

Mahammadali Zamanli

+994-55-397-75-00 | zamanli.mehemmedeli@gmail.com | [Linkedin](#) | [Github](#) | [Portfolio](#) | Baku, Azerbaijan

EDUCATION

ADA University

Bachelor of Science in Computer Science

Baku, Azerbaijan

Sep. 2022 – May 2027

Transport and Telecommunication Institute

B.S. in Computer Science (Exchange Program)

Riga, Latvia

Feb. 2025 – June 2025

TECHNICAL SKILLS

Languages: Java (Core/OOP), C (Systems), SQL (PostgreSQL), C# (Performance), MongoDB

Back-end: Spring WebFlux, Microservices, RabbitMQ, Redis, REST APIs, JUnit, Mockito

Tools & Concepts: Git (Collaborative Workflow), Docker, Maven, Linux (Bash), Reactive Programming

Competencies: High-Performance Computing, Distributed Systems, Asynchronous Messaging

EXPERIENCE

Software Engineer Intern (Performance & Distributed Systems)

July 2025 – Dec. 2025

AzSimX Azersilah

Baku, Azerbaijan

- Engineered high-performance logic in **C#**, optimizing rendering loops to achieve a 35% reduction in latency (14ms to 9ms), a methodology directly applicable to **Java server-side optimization**.
- Designed asynchronous synchronization modules between hardware and software, ensuring 100% data integrity in a **distributed systems architecture**.
- Utilized profiling tools to identify and eliminate memory leaks and processing bottlenecks in safety-critical simulation modules.

PROJECTS

Reactive Microservices Prototype | *Java, Spring WebFlux, RabbitMQ, MongoDB*

- Designed a non-blocking backend service using **Spring WebFlux** and **Project Reactor** to handle high-concurrency event streams with minimal resource overhead.
- Implemented asynchronous communication between services using **RabbitMQ** as a message broker, ensuring decoupled and scalable microservices architecture.
- Utilized **MongoDB** for flexible data storage of semi-structured event logs, achieving faster write speeds compared to traditional relational databases for high-velocity data.

Distributed Enterprise System | *Java, PostgreSQL, Redis, Maven*

- Architected a multithreaded Java system implementing strict **OOP design patterns** to manage concurrent resource allocation and data persistence.
- Integrated **Redis** for distributed caching, reducing database load and improving application response times for frequently accessed data clusters.
- Utilized **JUnit** and **Mockito** to implement comprehensive unit testing, ensuring zero-regression during the implementation of new service features.

Systems Engineering: High-Concurrency HTTP Server | *C, POSIX Sockets, Multithreading*

- Architected a multithreaded HTTP/1.1 server in C to master **Non-blocking I/O** and the TCP/IP stack, providing the foundation for understanding **Reactive Java Frameworks**.
- Implemented custom thread-pooling to manage 1,000+ simultaneous connections, mirroring the concurrency models used in modern high-performance Java backends.